- OffShare — Project Documentation
  - Table of Contents
  - 1. Project Overview
    - How It Works (High Level)
    - Key Features
  - 2. Why Expo + React Native?
    - The Problem
    - Why React Native?
    - Why Expo?
    - Why Not Pure Native (Kotlin/Jetpack Compose)?
  - 3. Why Kotlin for Native Modules?
    - The Technical Necessity
    - Why Kotlin Specifically (vs Java)?
  - 4. Kotlin Native Modules — Deep Dive
    - 4.1 FileServer.kt
    - 4.2 FileServerModule.kt
    - 4.3 FileServerPackage.kt
    - 4.4 HotspotManager.kt
    - 4.5 WifiConnectorModule.kt
    - 4.6 SafTransferModule.kt
    - 4.7 MediaScannerModule.kt
    - 4.8 MainApplication.kt
    - 4.9 MainActivity.kt
  - 5. JavaScript Screen Files — Deep Dive
    - 5.1 App.js
    - 5.2 HomeScreen.js
    - 5.3 ShareScreen.js
      - State Machine
      - Key Functions
      - QR Code Payload
      - UI Sections
      - Animations
    - 5.4 ReceiveScreen.js
      - State Machine
      - Key Functions
      - Utility Functions
      - UI Sections

        - Error Recovery
    - 6. Supporting Files
        - 6.1 theme.js
    - 7. Architecture Diagram
    - 8. Android Permissions
    - 9. Transfer Flow Summary
        - Sender Flow
        - Receiver Flow

OffShare — Project Documentation

Table of Contents

1. Project Overview
2. Why Expo + React Native?
3. Why Kotlin for Native Modules?
4. Kotlin Native Modules — Deep Dive
    - 4.1 FileServer.kt
    - 4.2 FileServerModule.kt
    - 4.3 FileServerPackage.kt
    - 4.4 HotspotManager.kt
    - 4.5 WifiConnectorModule.kt
    - 4.6 SafTransferModule.kt
    - 4.7 MediaScannerModule.kt
    - 4.8 MainApplication.kt
    - 4.9 MainActivity.kt
5. JavaScript Screen Files — Deep Dive
    - 5.1 App.js
    - 5.2 HomeScreen.js
    - 5.3 ShareScreen.js
    - 5.4 ReceiveScreen.js
6. Supporting Files
    - 6.1 theme.js
7. Architecture Diagram
8. Android Permissions

9. Transfer Flow Summary

---

1. Project Overview

OffShare is a **peer-to-peer, offline file transfer** application for Android. It allows two devices to exchange files over a direct Wi-Fi connection — *without internet, without cloud services, without Bluetooth*.

How It Works (High Level)

1. The **Sender** device creates a local Wi-Fi hotspot (`LocalOnlyHotspot` API) and starts an HTTP file server (NanoHTTPD) on port `3000`.
2. The Sender's screen displays a **QR code** containing the hotspot SSID, password, server IP, and port.
3. The **Receiver** scans the QR code, auto-connects to the Sender's hotspot, and downloads files directly over the LAN.
4. Files are saved to the Receiver's `Download/OffShare/` folder via Android's **Storage Access Framework (SAF)**.

Key Features

- 🚫 **No Internet Required** — Fully offline, peer-to-peer transfer
- 🔒 **Secure LAN Transfer** — Data stays on the local network, never leaves the device
- 📸 **QR Code Connection** — Zero manual configuration; scan and connect
- 📂 **Multi-File Support** — Select and transfer multiple files at once
- 📊 **Progress Tracking** — Real-time per-file and overall progress indicators
- 🔄 **Auto Wi-Fi Join** — Receiver automatically connects to the Sender's hotspot
- 💾 **SAF Integration** — Files saved to public Downloads, visible in the system file manager

---

2. Why Expo + React Native?

The Problem

OffShare needs to be an Android app that does heavy native operations (hotspot creation, Wi-Fi network joining, HTTP server, file system access) while also having a polished, modern UI with animations, state transitions, and real-time progress updates.

Why React Native?

Reason	Explanation
Rapid UI Development	React Native provides a component-based UI model with JSX, making it significantly faster to build complex screens (state-machine UIs, progress bars, animations) than native Android XML/Compose.
JavaScript for Business Logic	The transfer state machine, QR data encoding/decoding, permission flow orchestration, and UI state management are all complex logic tasks that benefit from JavaScript's flexibility and rapid iteration.
Native Bridge	React Native's `NativeModules` bridge allows us to call Kotlin code directly from JavaScript. This is critical because OffShare needs low-level Android APIs that don't have JavaScript equivalents.
Cross-Platform Potential	While OffShare is currently Android-only, React Native provides a path to iOS in the future without rewriting the UI layer.
Hot Reload	During development, React Native's Fast Refresh enables instant UI changes without rebuilding the APK — dramatically speeding up UI/UX iteration.

Why Expo?

Reason	Explanation
Managed Toolchain	Expo provides pre-configured build tools, Metro bundler, and development server out of the box. No manual Gradle/Metro/Babel configuration needed.

Reason	Explanation
Expo Modules	OffShare uses several Expo modules that provide battle-tested, well-maintained native functionality:`expo-camera` (QR scanning), `expo-document-picker` (file selection), `expo-file-system` (cache management, SAF access), `expo-location` (location services check).
Development Builds	With `expo-dev-client`, we run a custom development build that includes our Kotlin native modules while still getting the Expo developer experience (dev menu, error overlays, fast refresh).
Build System	`expo run:android` handles the full native build pipeline — Gradle, dexing, APK generation — with a single command.
New Architecture Support	The project has `newArchEnabled: true` in `app.json`, opting into React Native's New Architecture (Fabric renderer) for improved performance.

Why Not Pure Native (Kotlin/Jetpack Compose)?

While a pure native app could achieve the same functionality, React Native + Expo was chosen because:

- The UI layer (3 screens with complex state machines, animations, progress bars) would take **significantly longer** to build and iterate on in native Android.
- The core native logic (hotspot, server, Wi-Fi, SAF) is already written in Kotlin and accessed via the bridge — we get the **best of both worlds**.
- The Expo ecosystem provides ready-made solutions for camera (QR scanning), document picking, and file system access that would otherwise require manual implementation.

---

3. Why Kotlin for Native Modules?

The Technical Necessity

React Native and Expo provide excellent UI and many common native features, but **OffShare requires low-level Android system APIs** that have no JavaScript equivalent and no existing Expo/RN library support:

Feature	Android API Required	Why JS Can't Do This
Wi-Fi Hotspot	`WifiManager.startLocalOnlyHotspot()`	Android system API — requires direct access to `WifiManager` service
HTTP File Server	NanoHTTPD (JVM library)	Needed an in-process HTTP server that can serve files from the filesystem. No JS equivalent that runs embedded in an Android app.
Wi-Fi Network Joining	`WifiNetworkSpecifier` + `ConnectivityManager`	Programmatic Wi-Fi connection with process network binding requires Android `ConnectivityManager` APIs (API 29+)
SAF File Copy	`ContentResolver.openOutputStream()`	Writing to SAF `content://` URIs requires native `ContentResolver` access. JS-side base64 encoding would cause out-of-memory crashes on large files.
Media Scanner	`MediaScannerConnection.scanFile()`	Making downloaded files visible in the system Gallery/Files app requires triggering

Feature	Android API Required	Why JS Can't Do This
		the Android MediaScanner service
Network Interface Enumeration	`java.net.NetworkInterface`	Detecting the IP address of the hotspot interface requires iterating JVM `NetworkInterface` objects

Why Kotlin Specifically (vs Java)?

- **Kotlin is the official Android language** — Google recommends it for all new Android development.
- **Null safety** — Critical for code that deals with system callbacks that may return null (hotspot config, network interfaces).
- **Concise syntax** — Kotlin's `when` expressions, extension functions, and data classes make the native code significantly more readable.
- **Expo/RN ecosystem alignment** — Expo's own native modules and the React Native template both use Kotlin by default.

---

4. Kotlin Native Modules — Deep Dive

All native Kotlin files are located at:

```
android/app/src/main/java/com/seven1111/offshare/
```

4.1 `FileServer.kt`

Purpose: The core HTTP file server powered by NanoHTTPD.

What it does:

- Extends `NanoHTTPD` to create an embedded HTTP server that runs inside the Android app.
- Exposes three endpoints:
  - `GET /ping` — Health check. Returns `{"status": "ok"}`. Used by the Receiver to verify the Sender is reachable.
  - `GET /files` — Returns a JSON array of all shared files with their names and sizes. Also triggers the `onReceiverConnected` callback on first request (detects when a Receiver connects).
  - `GET /download?name=<filename>` — Serves a specific file as a binary download (`application/octet-stream`). URL-decodes the filename parameter to handle spaces and special characters.
- Handles file name matching by comparing the decoded `name` query parameter against the list of `File` objects provided at construction.

Why Kotlin is required: NanoHTTPD is a JVM library — it can only run in the JVM/Dalvik runtime, not in the JS engine. An in-process HTTP server is essential because the Receiver connects directly to this server over the LAN.

4.2 `FileServerModule.kt`

Purpose: The React Native bridge module that exposes the file server functionality to JavaScript.

Exposed methods (callable from JS via `NativeModules.FileServer`):

Method	Description
`startServer(port, filePaths)`	Starts the NanoHTTPD server on the given port, serving the specified file paths. Cleans up any previous server instance. Validates that files exist on disk before serving. Fires `onReceiverConnected` event when a Receiver hits the `/files` endpoint.
`stopServer()`	Stops the running server gracefully.
`getLocalIPv4()`	Detects the device's best local IPv4 address for the hotspot interface. Uses a sophisticated selection algorithm that prioritizes Wi-Fi/hotspot interfaces (`wlan`, `swlan`, `ap`, `softap`) and private LAN ranges

Method	Description
	(`192.168.x.x`, `10.x.x.x`, `172.16-31.x.x`). Excludes cellular (`rmnet`), VPN (`tun`), and loopback interfaces.
`dumpNetworkInterfaces()`	Debug utility that returns ALL network interfaces with their IPs, up/down status, and loopback flag. Used during development to diagnose IP detection issues.

Why Kotlin is required:

- `java.net.NetworkInterface` enumeration for IP detection is a JVM-only API.
- The module manages the lifecycle of the `FileServer` instance (start/stop) and bridges native events (`onReceiverConnected`) to JavaScript.

4.3 `FileServerPackage.kt`

Purpose: React Native package registration file. This is the glue that tells React Native about all custom native modules.

What it does:

- Implements `ReactPackage` interface.
- Registers all five custom native modules in `createNativeModules()`:
  1. `FileServerModule`
  2. `HotspotManager`
  3. `WifiConnectorModule`
  4. `MediaScannerModule`
  5. `SafTransferModule`
- This package is added to React Native's package list in `MainApplication.kt`.

Why it exists: React Native's module system requires a `ReactPackage` to discover and instantiate native modules. Without this file, none of the Kotlin modules would be accessible from JavaScript.

4.4 `HotspotManager.kt`

Purpose: Manages the Android `LocalOnlyHotspot` lifecycle — creating and destroying a Wi-Fi hotspot that other devices can connect to.

Exposed methods (callable from JS via `NativeModules.HotspotManager`):

Method	Description
`startHotspot()`	Calls `WifiManager.startLocalOnlyHotspot()` to create a temporary Wi-Fi hotspot. Waits 1500ms after the `onStarted` callback for the network interface to fully initialize. Returns the SSID and password (auto-generated by Android). Fires `onHotspotStarted` event to JS.
`stopHotspot()`	Closes the hotspot reservation, shutting down the hotspot. Fires `onHotspotStopped` event.
`isHotspotRunning()`	Returns whether a hotspot reservation is currently active.
`isLocationEnabled()`	Checks if GPS or Network location provider is enabled (required by Android before `LocalOnlyHotspot` can start).

Why Kotlin is required:

- `LocalOnlyHotspot` is an Android system API (`WifiManager`) that requires native access. There is no JavaScript or Expo equivalent.
- The callback-based API (`LocalOnlyHotspotCallback.onStarted`, `.onFailed`, `.onStopped`) must run on the Android main thread (`Handler(Looper.getMainLooper())`).
- Security exceptions (missing location permission) need to be caught and translated to user-friendly error messages.

Key implementation details:

- Uses `LocalOnlyHotspot` (not traditional tethering) — this is a modern API (Android 8.0+) that creates an isolated network specifically for app-to-app communication. It does NOT require the user to manually go into settings.
- The SSID and password are **auto-generated by Android** — the app cannot control them. This is why we use a QR code to transmit the credentials.
- The 1500ms delay after `onStarted` is intentional — the network interface needs time to fully bind and become routable.

4.5 `WifiConnectorModule.kt`

Purpose: Programmatically connects the Receiver device to the Sender's hotspot and binds all network traffic to that Wi-Fi connection.

Exposed methods (callable from JS via `NativeModules.WifiConnector`):

Method	Description
`connectToHotspot(ssid, password)`	Uses `WifiNetworkSpecifier` (Android 10+) to request a connection to the Sender's hotspot. After connecting, calls `cm.bindProcessToNetwork(network)` to force all HTTP traffic through the Wi-Fi connection (instead of mobile data). Includes a 30-second timeout.
`clearNetworkBinding()`	Unbinds the process from the Wi-Fi network, returning to default routing (mobile data).**Must** be called after transfer completes.
`isNetworkBound()`	Returns whether the process is currently bound to a specific network.

Why Kotlin is required:

- `WifiNetworkSpecifier` and `ConnectivityManager.requestNetwork()` are Android-only APIs (API 29+).
- **Process network binding** (`bindProcessToNetwork`) is critical: without it, Android would route HTTP requests through mobile data instead of the local hotspot, making the Sender's server unreachable.
- The `NetworkCallback` system (`onAvailable`, `onUnavailable`, `onLost`) is a native Android callback pattern that must be handled in Kotlin.

Key implementation details:

- Pre-flight checks verify all required permissions (`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`, `NEARBY_WIFI_DEVICES` on Android 13+) and that Location Services are enabled.
- `removeCapability(NET_CAPABILITY_INTERNET)` tells Android this network doesn't need internet access — without this, Android might reject the connection since `LocalOnlyHotspot` networks have no internet.

- Previous network callbacks are cleaned up before new requests to avoid resource leaks.

4.6 `SafTransferModule.kt`

Purpose: Performs native file copying from a temporary cache file to a Storage Access Framework (SAF) `content://` URI.

Exposed methods (callable from JS via `NativeModules.SafTransfer`):

Method	Description
`copyFileToContentUri(sourceUri, destContentUri)`	Reads from a source file (cache location) and writes to a SAF destination URI using a 256KB buffer. Handles both `file://` and `content://` source URIs. Returns the number of bytes copied.

Why Kotlin is required:

- SAF URIs (`content://com.android.externalstorage.documents/...`) can only be written to via `ContentResolver.openOutputStream()` — a native Android API.
- The **critical reason** this module exists: without it, the JS layer would have to read the entire file as a base64 string and write it via `expo-file-system`. For large files (videos, archives), this would cause **out-of-memory (OOM) crashes** because base64 encoding inflates file size by ~33% and loads the entire string into memory.
- The native streaming approach (256KB buffer) keeps memory usage constant regardless of file size.

Key implementation details:

- `openSourceInputStream` handles edge cases with URI encoding — Android may return paths with percent-encoded characters (e.g., `%2C` for commas). The method tries multiple path candidates to find the actual file on disk.
- Uses `linkedSetOf` to deduplicate path candidates while preserving order.

4.7 `MediaScannerModule.kt`

Purpose: Triggers the Android MediaScanner on downloaded files so they appear in the system file manager, Gallery, and other media apps.

Exposed methods (callable from JS via `NativeModules.MediaScanner`):

Method	Description
`scanFile(filePath)`	Scans a single file path. The file becomes visible in the system's media database.
`scanFiles(filePaths)`	Batch-scans multiple file paths at once. Resolves immediately — scanning happens asynchronously in the background.

Why Kotlin is required:

- `MediaScannerConnection.scanFile()` is an Android system API. Without triggering it, downloaded files would exist on disk but **would NOT appear** in the user's file browser or gallery until the next system-wide media scan (which could be hours later).

4.8 `MainApplication.kt`

Purpose: The Android Application class — the entry point for the entire app.

What it does:

- Extends `Application` and implements `ReactApplication`.
- Configures the `ReactNativeHost` with:
  - Expo's `ReactNativeHostWrapper` for module lifecycle management.
  - The `FileServerPackage()` registration (which brings in all 5 custom native modules).
  - New Architecture (Fabric) support based on build config.
- Initializes Expo's `ApplicationLifecycleDispatcher` for Expo module lifecycle hooks.

Why Kotlin is required: This is a mandatory Android application file that must be written in a JVM language. It's the bootstrap point where our custom `FileServerPackage` is registered with React Native.

4.9 `MainActivity.kt`

Purpose: The single Android Activity that hosts the React Native view.

What it does:

- Sets the app theme (`AppTheme`) before `onCreate` to support the splash screen.
- Registers the main React Native component name (`"main"`).
- Configures the `ReactActivityDelegate` with New Architecture (Fabric) support.
- Handles back button behavior (moves to background on Android S+ instead of finishing the activity).

Why Kotlin is required: Android requires at least one `Activity` — this is the container that React Native renders into. Uses Expo's `ReactActivityDelegateWrapper` for compatibility with Expo modules.

---

5. JavaScript Screen Files — Deep Dive

All screen files are located at:

```
src/screens/
```

5.1 `App.js`

Location: `./App.js` (root)

Purpose: Root application component and navigation controller.

What it does:

- Manages the current screen state using `useState` — one of: `'home'`, `'share'`, `'receive'`.
- Renders the appropriate screen component based on the current state:
  - `home` → `HomeScreen`
  - `share` → `ShareScreen`
  - `receive` → `ReceiveScreen`
- Sets the status bar style (light text) and background color.
- Passes navigation callbacks to child screens:
  - `onSelectRole` to `HomeScreen` — navigates to share or receive.
  - `onBack` to `ShareScreen` and `ReceiveScreen` — returns to home.

Design decision: Instead of using React Navigation (a routing library), the app uses a simple `switch` statement for navigation. This is intentional — the app only has 3 screens, so a full router would be unnecessary overhead.

5.2 `HomeScreen.js`

Location: `src/screens/HomeScreen.js`

Purpose: The landing screen — lets the user choose between **sending** or **receiving** files.

What it does:

- Displays the app branding (⚡ icon + "OffShare" title) at the top.
- Shows two large, tappable cards:
  - **"Share Files"** — with an upload icon. Tapping navigates to `ShareScreen`.
  - **"Receive Files"** — with a radar icon. Tapping navigates to `ReceiveScreen`.
- Uses the app's design system (`COLORS`, `STYLES` from `theme.js`) for consistent dark-themed styling.

UI elements:

- Cards have a dark background (`#161B22`), rounded corners, subtle borders (`#30363D`), and elevation shadows.
- Icons use Expo's `MaterialCommunityIcons` from `@expo/vector-icons`.

5.3 `ShareScreen.js`

Location: `src/screens/ShareScreen.js`

Purpose: The **Sender** flow — handles file selection, hotspot creation, server startup, QR code display, and transfer monitoring.

This is the most complex screen in the app. It implements a full state machine with 10 states.

State Machine

```
idle → filesSelected → serverStarting → hotspotStarting → hotspotReady →
networkReady → waitingForReceiver → connected → sending → completed
```

Any state can also transition to `error`.

Key Functions

Function	What It Does
`pickFiles()`	Opens Expo's document picker (`expo-document-picker`) for multi-file selection. Copies selected files to the app's cache directory (`offshare/`) to ensure stable file paths. Decodes URI-encoded paths (e.g., `%20` → spaces) for correct filesystem access.
`requestPermissions()`	Requests Android runtime permissions:`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`, and `NEARBY_WIFI_DEVICES` (Android 13+). Shows a settings redirect if permissions are denied.
`ensureLocationServices()`	Checks if GPS/Network Location is enabled via `HotspotManager.isLocationEnabled()`. Shows a blocking alert directing the user to Location Settings if disabled.
`startSharing()`	**The main flow orchestrator.** Executes sequentially: (1) request permissions → (2) check

Function	What It Does
	location services → (3) start `LocalOnlyHotspot` via `HotspotManager` → (4) start NanoHTTPD server via `FileServer` → (5) detect local IPv4 → (6) generate QR code payload.
`stopSharing()`	Stops the server and hotspot, resets all state.
`logNetworkInterfaces()`	Debug utility that calls `FileServer.dumpNetworkInterfaces()` and logs all network interfaces to the console.

QR Code Payload

The QR code encodes a JSON object:

```
{
  "ssid": "DIRECT-Xx-ANDROID-<hash>",
  "password": "<auto-generated>",
  "ip": "192.168.x.x",
  "port": 3000,
  "filesEndpoint": "/files",
  "downloadEndpoint": "/download",
  "deviceName": "OffShare Sender"
}
```

UI Sections

1. **Header** — Back button + "Share Files" title.
2. **Step Indicator** — Four dots (Files → Server → Hotspot → Receiver) showing progress through the flow.
3. **Status Banner** — Dynamic status text showing current operation.
4. **File Picker Card** — Shows selected files with names, sizes, and a "Choose Files" button. Supports adding/removing files.
5. **"Start Sharing" Button** — Triggers the full sharing flow.
6. **Processing Indicator** — Animated loading spinner shown during hotspot/server startup.
7. **QR Code Display** — Shows the scannable QR code, hotspot credentials (SSID + password), and server address. Includes a pulse animation while waiting for the Receiver.

8. **Transfer Progress** — Overall progress bar and percentage (shown during active transfer).
9. **Stop/Reset Button** — Red outlined button to cancel or start a new transfer.

Animations

- **Pulse animation** — `Animated.loop` scale animation (1 → 1.3 → 1) on the radar icon while waiting for receiver.
- **Fade-in animation** — QR code fades in with `Animated.timing` (opacity 0 → 1).

5.4 `ReceiveScreen.js`

Location: `src/screens/ReceiveScreen.js`

Purpose: The **Receiver** flow — handles QR scanning, Wi-Fi connection, file list fetching, downloading, and SAF storage.

This is the largest screen (1662 lines) and the most technically complex. It handles the full receiver pipeline from QR scan to file download.

State Machine

```
idle → scanning → scanned → connecting → connected → receiving → complete
Any state can transition to → error
```

Valid transitions are enforced by a `VALID_TRANSITIONS` map — invalid transitions are logged and blocked.

Key Functions

Function	What It Does
`openScanner()`	Requests camera permission via `expo-camera`, then opens a full-screen QR scanner using `CameraView`.
`handleBarCodeScanned({ data })`	Parses the scanned QR data using a robust multi-pass parser (`parseQRPayloadRobust`)

Function	What It Does
	that handles: raw JSON, double-stringified JSON, escaped strings, and quoted blobs. Validates the payload and extracts sender info. Auto-starts connection flow.
`beginConnection(senderInfo)`	**The connection orchestrator.** Executes: (1) check permissions → (2) check location services → (3) try direct ping (fast-path if already on same network) → (4) join Wi-Fi hotspot via `WifiConnector.connectToHotspot()` → (5) validate LAN address → (6) ping sender server (8s timeout) → (7) fetch file list. Includes fallback logic if Wi-Fi join fails.
`ensureSafDirectory()`	Gets the SAF directory for saving files. Uses a persisted URI cache (`saf_download_uri.txt`) so the user only picks the folder once. Pre-navigates to `Download/` folder.
`startReceiving()`	**The download orchestrator.** For each file: (1) download to temp cache via `expo-file-system` → (2) create destination file via SAF → (3) stream-copy to SAF via `SafTransfer.copyFileToContentUri()` → (4) trigger media scan → (5) clean up temp file. Updates per-file and overall progress throughout.
`retryConnection()`	Re-attempts the connection flow with the previously scanned sender info.
`resetTransfer()`	Clears all state, unbinds network, returns to idle.

Utility Functions

Function	What It Does
`fetchWithTimeout(url, options, timeout)`	`fetch` wrapper with a hard timeout using `AbortController`. Prevents the app from

Function	What It Does
	hanging on network requests. Default: 8 seconds.
`isValidLanAddress(ip)`	Validates that an IP is a private LAN address (192.168.x.x, 10.x.x.x, 172.16-31.x.x). Security check to prevent connecting to public IPs.
`validateQRPayload(parsed)`	Validates the QR payload has all required fields (`ip`, `port`, `filesEndpoint`) and that the IP is a valid private LAN address.
`parseQRPayloadRobust(raw)`	Multi-pass JSON parser with up to 5 unwrapping iterations. Handles corner cases from different QR scanner implementations that may double-encode the data.
`sanitizeFileName(name)`	Strips illegal filesystem characters (`<>:"/|?*`) from filenames.
`getSafBaseName(fileName)`	Extracts the base name (without extension) for SAF file creation, since SAF auto-appends the extension based on MIME type.
`getMimeType(fileName)`	Maps file extensions to MIME types for SAF file creation. Supports images, videos, audio, documents, and archives. Falls back to `application/octet-stream`.
`loadSavedSafUri()` / `saveSafUri(uri)`	Persists and loads the SAF directory URI to/from a cache file so the user doesn't have to re-pick the download folder every time.

UI Sections

1. **Header** — Back button + "Receive Files" title.
2. **Step Indicator** — Three dots (Scan → Connect → Receive).
3. **Idle State** — Large "Scan QR Code" button with feature badges ("Auto Wi-Fi Connect", "Secure LAN Transfer").
4. **Full-Screen QR Scanner** — Uses `expo-camera`'s `CameraView` with a custom overlay (corner brackets, title, hint text).
5. **Scanned State** — Brief transition showing "QR Scanned!" with a spinner.

6. **Connecting State** — Shows detailed connection progress (sub-steps like "Joining Wi-Fi...", "Pinging sender...", "Fetching files...") with sender network info.
7. **Error State** — Contextual error display with type-specific icons and hints:
    - `wifi` error → "Wi-Fi Connection Failed" + Wi-Fi troubleshooting tips
    - `ping` error → "Cannot Reach Sender" + network connectivity advice
    - `download` error → "Download Failed"
    - Retry and Scan Again buttons
8. **Connected State** — File list with names, sizes, total size, and "Download All Files" button.
9. **Receiving/Complete State** — Overall progress bar, per-file progress with download indicators, "New Transfer" button on completion.

Error Recovery

The screen has robust error recovery:

- **Retry** — Re-attempts the connection flow without requiring a new QR scan.
- **Scan Again** — Full reset back to idle for a fresh scan.
- **Fallback ping** — If Wi-Fi hotspot join fails, attempts a direct ping to the sender's IP (handles cases where devices are already on the same network).
- **Network binding cleanup** — Always clears the network binding on error, completion, or unmount to prevent routing issues.

---

6. Supporting Files

6.1 `theme.js`

Location: `src/constants/theme.js`

Purpose: Centralized design system with colors, fonts, and reusable styles.

Color Palette:

Token	Value	Usage
`background`	`#0B0F14`	Main app background (near-black)
`accent`	`#2F80FF`	Primary action color (blue)

Token	Value	Usage
`card`	`#161B22`	Card/surface background
`text`	`#FFFFFF`	Primary text (white)
`textSecondary`	`#8B949E`	Secondary/muted text
`success`	`#2EA043`	Success states (green)
`error`	`#DA3633`	Error states (red)
`border`	`#30363D`	Border/divider color

Shared Styles:

- `STYLES.card` — Reusable card style with rounded corners, shadow, and elevation.
- `STYLES.title` — Large bold white text (24px).
- `STYLES.subtitle` — Small muted text (14px).
- `STYLES.heading` — Medium semi-bold text (18px).

---

7. Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          JAVASCRIPT LAYER                            │
│                                                                      │
│  ┌──────────────┐     ┌──────────────────┐     ┌────────────────────┐      │
│  │ HomeScreen   │     │ ShareScreen      │     │  ReceiveScreen     │      │
│  │              │     │                  │     │                    │      │
│  │ • Role       │     │ • File Pick      │     │ • QR Scanner       │      │
│  │   Select     │     │ • Start Flow     │     │ • Wi-Fi Connect    │      │
│  │              │     │ • QR Display     │     │ • File Download    │      │
│  │              │     │ • Progress       │     │ • SAF Save         │      │
│  └──────────────┘     └──────────────────┘     └────────────────────┘      │
│        │                     │                      │                │
│  ┌──────┴──────────────────────┴──────────────────────┴──────────────┐    │
│  │                     App.js (Router)                              │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                      │
├──────────────────────── NativeModules Bridge ────────────────────────┤
│                                                                      │
│                          KOTLIN LAYER                                │
│                                                                      │
│  ┌────────────────────┐  ┌────────────────────┐                      │
│  │  HotspotManager    │  │  WifiConnector     │                      │
│  │  • start/stop      │  │  • connectToHotspot│                      │
```

```
│  │  • isLocationOn    │  │  • bindProcess      │                  │
│  │  • LocalOnlyAP     │  │  • clearBinding     │                  │
│  └────────────────────┘  └─────────────────────┘                  │
│                                                                   │
│  ┌────────────────────┐  ┌─────────────────────┐                  │
│  │  FileServerModule  │  │  FileServer         │                  │
│  │  • startServer     │  │  • NanoHTTPD        │                  │
│  │  • stopServer      │  │  • /ping            │                  │
│  │  • getLocalIPv4    │  │  • /files           │                  │
│  │  • dumpInterfaces  │  │  • /download        │                  │
│  └────────────────────┘  └─────────────────────┘                  │
│                                                                   │
│  ┌────────────────────┐  ┌─────────────────────┐                  │
│  │  SafTransfer       │  │  MediaScanner       │                  │
│  │  • copyToSAF       │  │  • scanFile         │                  │
│  │  • stream 256KB    │  │  • scanFiles        │                  │
│  └────────────────────┘  └─────────────────────┘                  │
│                                                                   │
│  ┌──────────────────────────────────────────────────────────┐     │
│  │  FileServerPackage — registers all modules               │     │
│  │  MainApplication — bootstraps React Native + Expo        │     │
│  │  MainActivity — hosts the React Native view              │     │
│  └──────────────────────────────────────────────────────────┘     │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

---

8. Android Permissions

The following permissions are declared in `AndroidManifest.xml`:

Permission	Purpose
`INTERNET`	NanoHTTPD server needs to accept incoming TCP connections
`ACCESS_WIFI_STATE`	Check current Wi-Fi state
`CHANGE_WIFI_STATE`	Start `LocalOnlyHotspot`
`ACCESS_NETWORK_STATE`	Check network connectivity
`ACCESS_FINE_LOCATION`	Required by Android to start `LocalOnlyHotspot` and use `WifiNetworkSpecifier`
`ACCESS_COARSE_LOCATION`	Backup location permission
`NEARBY_WIFI_DEVICES`	Required on Android 13+ for Wi-Fi operations

Permission	Purpose
`CAMERA`	QR code scanning on receiver side
`READ_EXTERNAL_STORAGE`	File access on Android ≤ 12
`WRITE_EXTERNAL_STORAGE`	File writing on Android ≤ 8
`READ_MEDIA_IMAGES/VIDEO/AUDIO`	Scoped media access on Android 13+
`VIBRATE`	Haptic feedback
`SYSTEM_ALERT_WINDOW`	Dev menu overlay (debug builds)

Additional manifest settings:

- `android:usesCleartextTraffic="true"` — Required because the LAN HTTP server uses `http://` (not HTTPS). This is safe because the traffic never leaves the local network.
- `android:screenOrientation="portrait"` — Locked to portrait mode.

---

9. Transfer Flow Summary

Sender Flow

```
1. User opens app → HomeScreen
2. Taps "Share Files" → ShareScreen
3. Taps "Choose Files" → Document picker opens
4. Selects files → Files copied to cache
5. Taps "Start Sharing":
   a. [Permission check] → Request Location + Nearby Devices
   b. [Location check]   → Ensure GPS is ON
   c. [Hotspot start]    → LocalOnlyHotspot creates Wi-Fi AP
   d. [Server start]     → NanoHTTPD starts on port 3000
   e. [IP detection]     → Find hotspot interface IP address
   f. [QR generation]    → Encode SSID + password + IP + port
6. QR code displayed → Waiting for receiver
7. Receiver scans QR → "onReceiverConnected" event fires
8. Receiver downloads files → Progress updates
9. Transfer complete → "New Transfer" button shown
```

Receiver Flow

```
1. User opens app → HomeScreen
2. Taps "Receive Files" → ReceiveScreen
3. Taps "Scan QR Code":
   a. [Camera permission] → Request if needed
   b. [Scanner opens]     → Full-screen camera view
4. Scans sender's QR code:
   a. [Parse QR]          → Multi-pass JSON unwrapping
   b. [Validate payload]  → Check IP, port, endpoints
5. Auto-connection begins:
   a. [Check permissions]  → Location + Nearby Devices
   b. [Check location ON]  → Ensure GPS is enabled
   c. [Fast-path check]    → Try direct ping (already connected?)
   d. [Join hotspot]       → WifiNetworkSpecifier request
   e. [Bind network]       → Force traffic through Wi-Fi
   f. [Ping sender]        → GET /ping → {"status": "ok"}
   g. [Fetch file list]    → GET /files → [{name, size}, ...]
6. File list displayed → "Download All Files" button
7. User taps download:
   a. [SAF directory]     → Pick/recall download folder
   b. For each file:
      i.   Download to temp cache (expo-file-system)
      ii.  Create SAF file (content:// URI)
      iii. Stream copy to SAF (native 256KB buffer)
      iv.  Trigger media scanner
      v.   Cleanup temp file
   c. Update progress throughout
8. Transfer complete → Alert with file count + save location
9. Network binding cleared → Device returns to normal networking
```

This documentation was auto-generated from the OffShare source code on February 13, 2026.